

DAYANANDA SAGAR UNIVERSITY

(School of Computer Applications)

PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Computer Applications (BCA)

ASSIGNMENT TRACKING SYSTEM

Submitted By

Asif Saleem

Akshaya M

Giri Balaji C

Field	Details
Course	2nd Semester BCA
Course Code	25CA1202
Component	MSE1: Foundational Project
Project Guide	Miss Priyanka Soni, Assistant Professor
Academic Year	2025-26

DECLARATION

We hereby declare that the project report entitled "Assignment Tracking System" is a record of the work carried out by us during the academic year 2025-2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) from Dayananda Sagar University.

The project has been developed under the guidance of Mr. Chandaka Giri Babu, Assistant Professor, School of Computer Applications. We confirm that this work is original and has not been submitted to any other university or institution for the award of any degree or diploma.

We further state that all sources of information and tools used in this project have been properly acknowledged.

Place: Bengaluru

Date: 07 May 2026

1. Assif Saleem

Signature: _____

2. Akshaya M

Signature: _____

3. Giri Balaji C

Signature: _____

CERTIFICATE

This is to certify that the project report entitled "Assignment Tracking System" is a bonafide record of the work carried out by the following students of 2nd Semester Bachelor of Computer Applications (BCA), Dayananda Sagar University, during the academic year 2025-2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA).

The project has been completed under my guidance and supervision. To the best of my knowledge, this work is original and has not been submitted previously for the award of any degree or diploma in any university.

Student Details

Sl. No.	Student Name	Signature
1	Assif Saleem	_____
2	Akshaya M	_____
3	Giri Balaji C	_____

Project Guide	Head of the Department
Signature: _____	Signature: _____
Name: _____	Name: _____
Designation: _____	Designation: _____

ABSTRACT

The Assignment Tracking System is a web-based application designed to simplify the process of managing academic assignments, submissions, deadlines, and grading. In many academic environments, students and faculty members still depend on manual reminders, spreadsheets, or separate communication channels to track assignment work. This can lead to missed deadlines, unclear submission status, and difficulty in reviewing academic progress.

The proposed system provides a centralized platform for assignment management. It includes role-based login for students and faculty members, assignment creation, student-specific views, submission tracking, grading, search, filtering, sorting, overdue identification, and persistent browser-based storage. Students can view pending assignments, submit work, record comments, and check grades. Faculty users can add assignments, monitor submissions, grade work, and delete outdated records.

The application is developed using HTML, CSS, and JavaScript. The system uses browser `localStorage` and `sessionStorage` to maintain login state and preserve assignment data in the prototype version. The interface is responsive and uses a black and yellow visual theme that matches the login page. The system is suitable as a foundational academic project because it demonstrates user authentication flow, role-based dashboards, data handling, form validation, DOM rendering, and client-side persistence.

Overall, the Assignment Tracking System reduces manual tracking effort, improves deadline visibility, and provides a structured workflow for students and faculty. It can be extended in the future with a backend database, real file uploads, notifications, analytics, and institutional integration.

TABLE OF CONTENTS

Section	Page
Declaration	2
Certificate	3
Abstract	4
Chapter 1: Introduction	6
Chapter 2: Literature Survey	10
Chapter 3: System Analysis and Methodology	12
Chapter 4: System Design	14
Chapter 5: Implementation and Results	19
Chapter 6: Testing	22
Chapter 7: Conclusion and Future Enhancements	25
Appendix A: User Manual	26
Appendix B: Technical Documentation	29
Appendix C: Deployment and Maintenance	32
Appendix D: Sample Output Explanation	34
Appendix E: Viva Voce Preparation	36
References	38

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Assignment Tracking System is a web-based application developed to organize and simplify the assignment lifecycle in an academic setting. It provides separate student and faculty dashboards so that each user can perform tasks according to their role. The student dashboard focuses on viewing assignments, tracking due dates, submitting work, and checking grades. The faculty dashboard focuses on creating assignments, viewing student submissions, grading assignments, and monitoring overall progress.

The project is implemented as a frontend application using HTML, CSS, and JavaScript. It uses the browser's sessionStorage and localStorage features to store login details and assignment data in the prototype. This makes the project easy to run without server installation while still demonstrating realistic data operations such as adding, updating, filtering, deleting, and persisting records.

1.2 Problem Statement

Students often find it difficult to keep track of multiple assignments, deadlines, submission status, and grades when information is shared through different channels. Faculty members also face difficulty when monitoring submissions and manually checking which assignments need grading. Manual tracking can result in missed deadlines, duplicate records, incomplete feedback, and time-consuming follow-up.

The project addresses these problems by creating a single browser-based system where assignments can be created, viewed, submitted, graded, and filtered efficiently.

1.3 Objectives of the Project

- To provide a simple login-based system for students and faculty members.
- To allow faculty users to add, view, grade, and delete assignment records.
- To allow students to view assigned work and submit assignment details.
- To display assignment statistics such as pending, submitted, graded, and overdue counts.
- To support search, status filtering, sorting, and responsive display of assignment data.
- To store project data in browser storage for prototype-level persistence.

1.4 Scope of the Project

The scope of this project is limited to a browser-based academic assignment management prototype. It includes login, role selection, assignment data management, submission status tracking, grading, and dashboard statistics. It is suitable for small classroom demonstrations and foundational project evaluation.

The current version does not include a backend server, institutional database, email notifications, cloud deployment, or actual file upload storage. These features are identified as future enhancements.

1.5 Software and Hardware Requirements

Category	Requirement
Frontend	HTML5, CSS3, JavaScript
Storage	Browser sessionStorage and localStorage
Editor	Visual Studio Code or any code editor
Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox
Hardware	Computer with 4GB RAM or above
Operating System	Windows, Linux, or macOS

1.6 Functional and Non-Functional Requirements

Type	Requirements
Functional	Login, role selection, assignment creation, assignment submission, grading, deletion, filtering, sorting, statistics, and details view.
Non-Functional	Usability, responsiveness, data accuracy, fast page interaction, readable interface, and simple maintainability.

1.7 Domain Profile

The project belongs to the education technology domain. Assignment management systems are used in schools, colleges, and universities to improve communication between students and faculty members. Such systems help academic institutions reduce manual effort and improve transparency in coursework tracking.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Systems and Related Work

Existing assignment management approaches include manual notebooks, classroom announcements, spreadsheet trackers, email submissions, learning management systems, and cloud-based academic portals. Manual and spreadsheet-based approaches are simple but require repeated updates and do not provide automated status visibility. Learning management systems provide advanced features but may be complex for small classroom projects or basic academic demonstrations.

2.2 Limitations in Existing Systems

- Manual systems are slow and prone to missed deadlines.
- Spreadsheet systems are difficult to maintain when assignment volume increases.
- Email-based submission lacks centralized status tracking.
- Large learning platforms may be too complex for small projects.
- Some systems do not clearly separate student and faculty workflows.

2.3 Overview of Technologies Used

The project uses HTML for page structure, CSS for visual styling and responsive layout, and JavaScript for application logic. Browser sessionStorage is used to maintain login state during a session, while localStorage stores assignment records and remembered login details. This technology stack allows the project to run directly in a browser without a server.

2.4 Comparison of Approaches

Approach	Advantages	Limitations
Manual Record	Simple and no technical setup	Time-consuming, no automation, easy to lose data
Spreadsheet	Organized rows and columns	Limited role access, manual updates, no interactive workflow
Email Submission	Easy file sharing	Difficult tracking and grading overview
Assignment Tracking System	Role-based dashboard, status tracking, filtering, grading, and persistence	Prototype storage only; backend needed for production

CHAPTER 3: SYSTEM ANALYSIS AND METHODOLOGY

3.1 Existing System

The existing system for many classrooms relies on verbal announcements, written lists, or spreadsheets. These methods require manual follow-up and do not provide a dedicated dashboard for students and faculty. Students may not always know which assignments are pending, and faculty members may not have a quick view of submissions that need grading.

3.2 Proposed System

The proposed Assignment Tracking System provides a role-based web interface where the login page authenticates a user as a student or faculty member. After login, the user is redirected to the dashboard. Students can view their own assignments and submit them. Faculty users can view all assignments, add new assignments, grade submitted work, and delete records.

3.3 Methodology

1. Requirement identification: Understand the needs of students and faculty.
2. Interface design: Create a login page and dashboard layout.
3. Role handling: Store role, user name, and ID in browser storage after login.
4. Feature implementation: Add assignment creation, submission, grading, search, filtering, sorting, and statistics.
5. Validation and testing: Test valid and invalid login, student workflow, faculty workflow, and responsive layout.
6. Improvement: Add persistence, overdue status, safer rendering, and reset demo data.

3.4 Feasibility Study

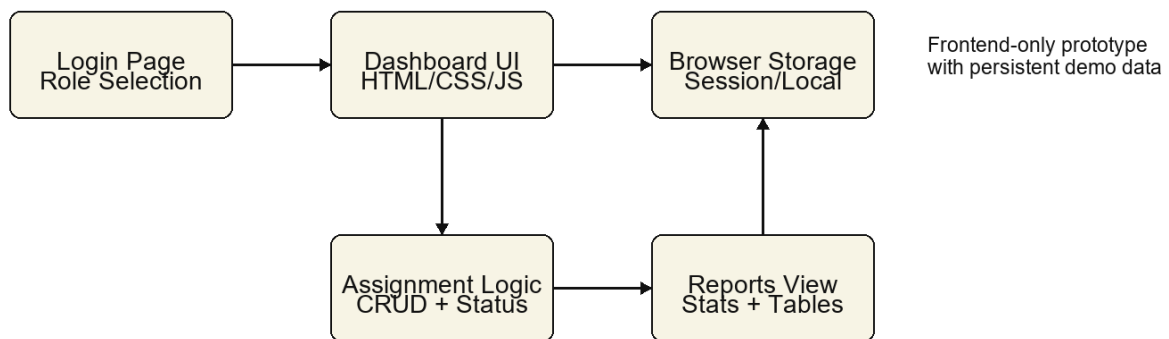
Feasibility Type	Analysis
Technical	The system uses common web technologies and runs in a standard browser.
Economic	No paid software or hosting is required for the prototype.
Operational	The interface is simple, role-based, and easy for students and faculty to understand.
Schedule	The project can be completed within a foundational project timeline because it is frontend-only.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

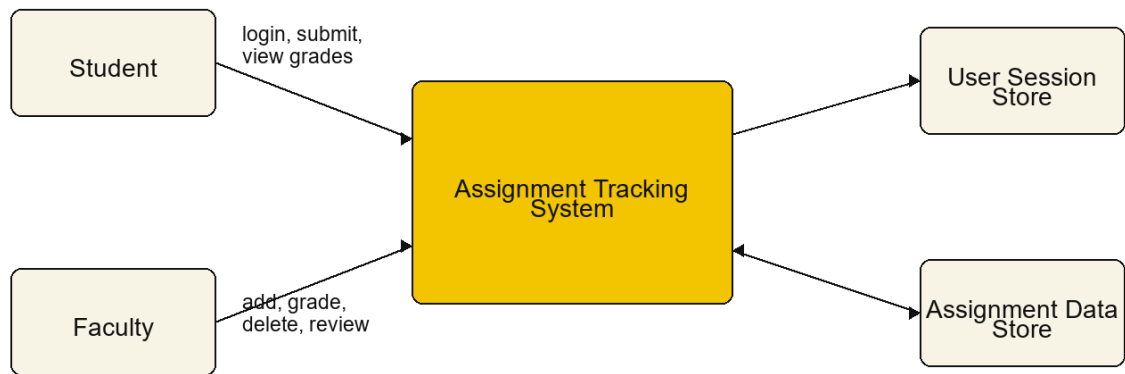
The project follows a simple client-side architecture. The presentation layer is built using HTML and CSS. The application logic is handled using JavaScript. Data is stored in browser storage for the prototype. Since there is no backend server in the current version, the application can run directly from local files.

Figure 4.1: System Architecture



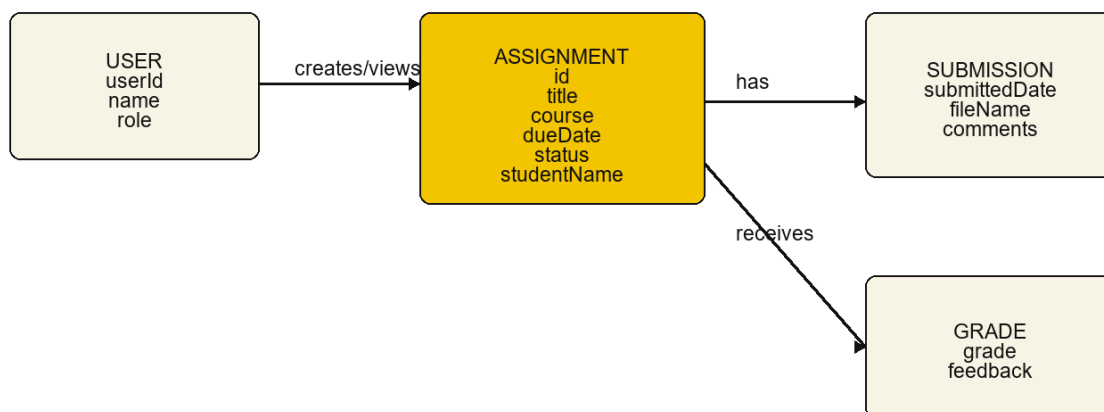
4.2 Data Flow Diagram

The Data Flow Diagram shows how student and faculty actions are processed by the system. Login details are stored as session data, while assignment records are stored in the browser data store.

Figure 4.2: Data Flow Diagram

4.3 Database Design

The current prototype uses browser localStorage instead of a relational database. However, the data model can be mapped to database tables in a future backend version. The main entities are User, Assignment, Submission, and Grade.

Figure 4.3: Entity Relationship Model

4.3.1 Schema Design

Entity	Important Fields	Purpose
User	userId, name, role	Stores login identity and role.
Assignment	id, title, course, studentName, dueDate, status, description	Stores assignment details and ownership.
Submission	submittedDate, submissionFileName, submissionComments	Stores submission metadata in the prototype.
Grade	grade, feedback	Stores faculty evaluation details.

4.4 User Interface Design

The user interface contains a login page and a dashboard page. The login page allows the user to choose Student or Faculty, enter username and password, and optionally remember the session. The dashboard page displays statistics cards, controls for search/filter/sort, assignment tables, and modals for adding, submitting, grading, and viewing assignment details.

Screen	Main Controls	User Role
Login Page	Role selector, username, password, remember me, login button	Student and Faculty
Student Dashboard	Stats, assignment list, submit button, view details	Student
Faculty Dashboard	Stats, add assignment, grade, view, delete, filters	Faculty
Modal Windows	Add form, submit form, grade form, detail view	Role-based

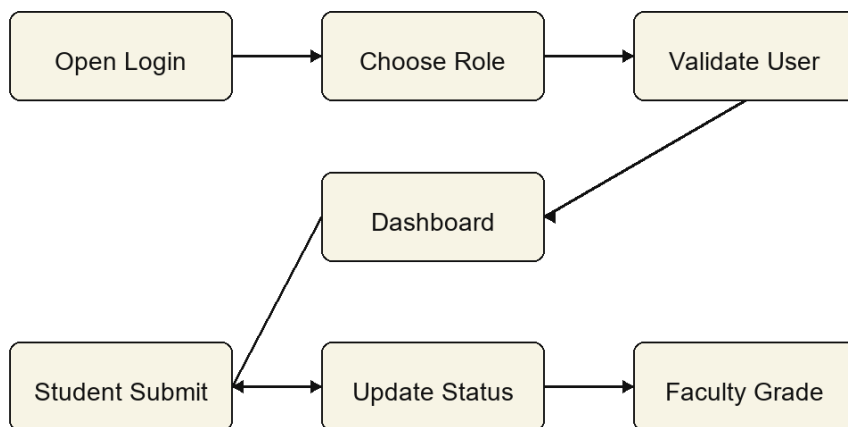
4.5 Module Description

Module	Description
Authentication Module	Validates demo credentials, stores user role and redirects to dashboard.
Dashboard Module	Displays role-specific statistics and assignment tables.
Assignment Module	Adds assignments and stores them with student, course, due date, and description.
Submission Module	Records submission time, file name, comments, and late/submitted status.

Module	Description
Grading Module	Allows faculty users to enter marks and feedback.
Search and Filter Module	Filters assignments by keyword and status and sorts records by due date, title, or status.
Storage Module	Uses localStorage and sessionStorage for prototype persistence.

4.6 Activity Diagram

Figure 4.4: Activity Flow



CHAPTER 5: IMPLEMENTATION AND RESULTS

5.1 Implementation Environment

The system was implemented as a frontend web application. The pages are stored as HTML files and can be opened through a browser. The login page uses external CSS from style.css, while the tracker page contains its own CSS and JavaScript for dashboard behavior.

Tool / Technology	Purpose
HTML5	Defines login page, dashboard structure, tables, and forms.
CSS3	Provides visual design, layout, responsiveness, and black/yellow theme.
JavaScript	Handles authentication, rendering, validation, status updates, and storage.
sessionStorage	Stores temporary login state.
localStorage	Stores remembered login and assignment data.
Browser	Runs and tests the application.

5.2 Implementation of Modules

5.2.1 Login Module

The login module lets the user choose either the student role or faculty role. It validates username and password against demo credential arrays. If login succeeds, userRole, userName, userId, and isLoggedIn are stored in browser storage and the user is redirected to the tracker page.

5.2.2 Assignment Dashboard Module

The dashboard reads the logged-in user role and renders different views. A student sees only assignments assigned to their name. A faculty user sees all assignments and can perform grading or deletion.

5.2.3 Submission and Grading Modules

Students can submit pending assignments by entering optional comments and selecting a file. The prototype stores only the file name because a real backend is needed for actual file upload storage. Faculty users can grade submitted or late assignments by entering marks out of 100 and feedback.

5.2.4 Storage and Persistence

Assignment records are saved in `localStorage` under a versioned storage key. This allows assignment changes to remain available after page refresh. A Reset Data button restores the demo data set.

5.3 Core Functionalities

- Role-based student and faculty dashboards.
- Assignment creation with title, course, due date, student name, and description.
- Student submission workflow with file name and comments.
- Faculty grading workflow with marks and feedback.
- Automatic submitted or late status based on due date.
- Overdue display for pending assignments past the due date.
- Search, status filter, sorting, and assignment details modal.
- Responsive layout for desktop and mobile screens.

5.4 Output and Results

Feature	Result
Login	Valid credentials redirect the user to the assignment dashboard.
Student View	Only the logged-in student's assignments are displayed.
Faculty View	All assignment records are displayed with grading and delete actions.
Assignment Add	New records appear instantly and remain after refresh.
Submission	Status changes to submitted or late depending on due date.
Grading	Grade and feedback are saved and shown in the detail view.
Filters	Search, status filter, and sorting update the table immediately.

5.5 Performance Evaluation

The application performs quickly because all processing is handled in the browser and the data set is small. Rendering is immediate for the prototype use case. The project demonstrates efficient DOM updates, simple validation, and lightweight client-side storage.

CHAPTER 6: TESTING

6.1 Testing Strategies

Testing was carried out to verify that each module works as expected. Unit testing was applied to individual functions such as role selection, login validation, date formatting, overdue detection, assignment saving, submission, and grading. Integration testing was used to ensure that login state, dashboard rendering, and assignment storage work together correctly.

6.1.1 Unit Testing

Individual features such as login form validation, password visibility toggle, assignment form submission, grade validation, and storage saving were checked independently.

6.1.2 Integration Testing

The complete flow from login to dashboard, assignment creation, student submission, faculty grading, and final details view was tested to ensure correct data flow.

6.1.3 System Testing

The application was tested in a browser with both student and faculty demo accounts. The responsive layout was checked by reducing the browser width and verifying that controls and tables remained usable.

6.2 Test Cases

ID	Module	Input / Action	Expected Output	Result
TC01	Login	Valid student credentials	Student dashboard opens	Pass
TC02	Login	Invalid password	Error message is displayed	Pass
TC03	Role	Select faculty role	Faculty role becomes active	Pass
TC04	Assignment	Faculty adds assignment	Assignment appears in dashboard	Pass
TC05	Student View	Login as assigned student	Only own assignments are visible	Pass
TC06	Submission	Submit pending assignment	Status changes to submitted or late	Pass
TC07	Grading	Faculty enters grade 92	Assignment status becomes graded	Pass
TC08	Filter	Select Overdue	Only overdue assignments appear	Pass
TC09	Persistence	Refresh after adding data	Saved data remains available	Pass
TC10	Responsive UI	Open on small screen	Controls stack cleanly	Pass

6.3 Bug Fixing and Improvements

- Fixed student filtering so student-created assignments belong to the logged-in student.

- Added localStorage persistence so assignment data does not disappear after refresh.
- Added safer HTML escaping for user-entered fields.
- Improved status handling by adding overdue display.
- Improved layout with responsive controls and scrollable tables.
- Replaced alert-only details with a structured details modal.

CHAPTER 7: CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The Assignment Tracking System successfully provides a structured way to manage academic assignments in a browser-based environment. It supports separate workflows for students and faculty members, making assignment creation, submission, grading, and review more organized. The system improves visibility of due dates and assignment statuses while reducing the need for manual tracking.

The project demonstrates important frontend development concepts such as forms, validation, role-based rendering, storage, dynamic tables, modals, responsive design, and event-driven JavaScript.

7.2 Limitations

- The prototype does not use a backend server or database.
- File upload stores only the file name, not the actual file.
- Authentication uses demo credentials and is not production-secure.
- Data is browser-specific and not shared across devices.
- Notifications and email reminders are not included.

7.3 Future Enhancements

- Add Node.js or Express backend with a MySQL database.
- Add real file upload and download support.
- Add email or SMS deadline reminders.
- Add admin panel for managing courses, students, and faculty.
- Add analytics charts for submission trends and grading performance.
- Deploy the application online for multi-user access.
- Add stronger authentication and password encryption.

APPENDIX A: USER MANUAL

A.1 Purpose of the User Manual

This user manual explains how students and faculty members can use the Assignment Tracking System. It is written for first-time users who need to understand the login process, dashboard controls, assignment submission, grading workflow, and basic troubleshooting.

The application is designed as a simple browser-based prototype. It can be opened from the index.html file, and after successful login the user is redirected to the assignment-tracker.html dashboard.

A.2 Demo Login Credentials

Role	Username	Password	Name Displayed
Student	student	student123	John Doe
Student	student1	pass123	Jane Smith
Student	john.doe@university.edu	student123	John Doe
Faculty	faculty	faculty123	Dr. Sarah Johnson
Faculty	faculty1	pass123	Prof. Michael Brown
Faculty	sarah.johnson@university.edu	faculty123	Dr. Sarah Johnson

A.3 Student Workflow

- Open the login page in a supported browser.
- Select the Student role from the role selector.
- Enter valid student credentials and click Login.
- View the student dashboard and check the assignment statistics.
- Use the search box or status filter to locate a specific assignment.
- Click Submit for a pending assignment.
- Select a file if required and enter submission comments.
- Click Submit to update the assignment status.
- Open the View action to check details, submission status, grade, and feedback.

A.4 Faculty Workflow

- Open the login page and select the Faculty role.
- Enter valid faculty credentials and log in.
- Review total assignments, needs grading, overdue pending work, and average grade statistics.
- Click Add Assignment to create a new assignment for a student.
- Enter assignment title, course, due date, student name, and description.
- Use filters and sorting to review assignment records.

22. Click Grade for submitted or late assignments.
23. Enter marks out of 100 and feedback, then save the grade.
24. Use Delete only for outdated or incorrect records.

A.5 Dashboard Controls

Control	Purpose	Available To
Search Box	Find assignments by title, course, student name, or description.	Student and Faculty
Status Filter	Display assignments by pending, overdue, submitted, late, graded, or all status.	Student and Faculty
Sort Dropdown	Arrange assignments by due date, title, or status.	Student and Faculty
Add Assignment	Create a new assignment record.	Faculty; student can add personal demo assignment
Submit	Record assignment submission details.	Student
Grade	Enter marks and feedback for submitted work.	Faculty
View	Open complete assignment details.	Student and Faculty
Reset Data	Restore the original demo assignment set.	Student and Faculty

A.6 Common User Messages

The system uses toast messages and form validation to guide users. For example, when an assignment is saved, submitted, deleted, or graded, a short confirmation message appears. If required information is missing, the system prevents the operation and displays a message.

Situation	System Response
Invalid login credentials	Displays an error message and remains on the login page.
Assignment saved	Displays confirmation and updates the table immediately.
Assignment submitted	Records submission date and changes status.
Grade outside 0-100	Blocks saving and requests a valid grade.
No matching assignments	Displays an empty-state message.

A.7 Troubleshooting Guide

Problem	Possible Reason	Solution
Dashboard does not open	User is not logged in	Return to index.html and log in again.

Problem	Possible Reason	Solution
Assignment not visible to student	Assignment is assigned to a different student name	Faculty should verify the student name.
Changes disappeared on another browser	Data is stored locally in one browser	Use the same browser or add backend storage in future.
File not downloadable	Prototype stores only file name	Implement backend upload storage in future version.
Old demo data needed	Data has been modified	Click Reset Data on dashboard.

APPENDIX B: TECHNICAL DOCUMENTATION

B.1 File Structure

The project is organized as a small frontend application. The main files are `index.html`, `style.css`, `assignment-tracker.html`, and the `media` folder. The login page links to `style.css`, while the dashboard page contains its own CSS and JavaScript.

File / Folder	Description
<code>index.html</code>	Login page with role selection, demo credentials, password toggle, and redirect logic.
<code>style.css</code>	Stylesheet for the login page, including black/yellow theme and responsive layout.
<code>assignment-tracker.html</code>	Main dashboard page with assignment data, forms, modals, status logic, and storage.
<code>media/</code>	Contains favicon and supporting image assets used by the project.

B.2 Important JavaScript Functions

Function / Area	Purpose
<code>selectRole(role)</code>	Updates the selected login role and active role UI.
<code>togglePassword()</code>	Switches password field between hidden and visible text.
<code>checkAuth()</code>	Checks whether the user is logged in before opening dashboard.
<code>renderStats()</code>	Calculates and displays dashboard statistics based on role.
<code>renderTable()</code>	Renders student or faculty assignment table.
<code>openAddModal()</code>	Opens assignment form and configures fields by role.
<code>saveAssignments()</code>	Stores assignment data in <code>localStorage</code> .
<code>viewSubmission(id)</code>	Displays assignment, submission, grade, and feedback details.
<code>isOverdue(assignment)</code>	Detects pending assignments whose due date has passed.

B.3 Data Dictionary

Field	Data Type	Description
id	Number	Unique assignment identifier.
title	String	Assignment title entered by faculty or student.
course	String	Course or subject code/name.
studentName	String	Student who owns the assignment record.
dueDate	DateTime String	Deadline selected using datetime-local input.
status	String	pending, submitted, late, or graded.
submittedDate	DateTime String	Date and time when assignment was submitted.
description	String	Assignment instructions or notes.
grade	Number / null	Marks out of 100 after grading.
feedback	String	Faculty feedback after grading.
submissionFileName	String	Selected file name stored for prototype display.
submissionComments	String	Comments entered during submission.

B.4 Validation Rules

Input	Validation Rule	Reason
Username	Required	Needed to identify the user.
Password	Required	Needed for login validation.
Assignment Title	Required, limited length	Prevents empty or oversized records.
Course	Required	Every assignment must belong to a course.
Due Date	Required	Needed for pending, overdue, submitted, and late status.
Student Name	Required for faculty assignment creation	Links assignment to a student dashboard.
Grade	Required for grading, 0 to 100	Ensures valid academic marks.
Description / Comments	Optional, limited length	Allows notes without overly long entries.

B.5 Status Calculation Logic

The assignment status is calculated according to the assignment lifecycle. New assignments start as pending. If the current date is greater than the due date and the assignment is still pending, the dashboard displays it as overdue. When a student submits the assignment before the due date, the status becomes submitted. If submission happens after the due date, the status becomes late. After faculty grading, the status becomes graded.

Condition	Displayed Status
New assignment before due date	Pending
Pending assignment after due date	Overdue
Submitted before due date	Submitted
Submitted after due date	Late
Faculty saves grade	Graded

B.6 Security Considerations

The current system is a prototype and uses demo credentials stored in JavaScript. This is acceptable for a foundational project demonstration but not for production. A production version

should move authentication to a backend server, hash passwords, use secure sessions, validate all inputs on the server, and protect file uploads.

- Use server-side authentication and authorization.
- Store passwords using secure hashing algorithms.
- Validate and sanitize input on both client and server.
- Store uploaded files in a controlled location.
- Protect APIs using session tokens or JSON Web Tokens.
- Use HTTPS when deployed online.

APPENDIX C: DEPLOYMENT AND MAINTENANCE

C.1 Local Execution Steps

25. Place all project files in one folder.
26. Ensure index.html, style.css, assignment-tracker.html, and media folder are present.
27. Open index.html in Google Chrome or any modern browser.
28. Select Student or Faculty role and enter demo credentials.
29. Use the dashboard features according to the selected role.
30. Refresh the browser to confirm that assignment data remains stored.
31. Use Reset Data when the original demo records are required again.

C.2 Browser Compatibility

The application uses standard HTML, CSS, JavaScript, localStorage, and sessionStorage APIs. These are supported by modern browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari. For the best experience, a current version of Chrome or Edge is recommended.

C.3 Maintenance Plan

Maintenance Area	Recommended Action	Frequency
Demo Credentials	Review and update usernames/passwords for classroom demo.	Before presentation
Assignment Data	Reset demo data or clear localStorage if testing becomes cluttered.	As needed
UI Testing	Check layout on desktop and mobile widths.	After every UI change
Code Review	Review JavaScript functions for duplication or unsafe rendering.	After feature additions
Backup	Keep a backup before major edits.	Before major change

C.4 Suggested Backend Upgrade Plan

To convert the prototype into a multi-user production system, the application can be upgraded with a Node.js backend and MySQL database. The frontend can call backend APIs to login, create assignments, submit work, upload files, and save grades.

API Endpoint	Method	Purpose
/api/login	POST	Validate user credentials and create session.
/api/assignments	GET	Fetch assignments based on user role.
/api/assignments	POST	Create a new assignment.
/api/assignments/:id/submit	POST	Upload/record assignment submission.

API Endpoint	Method	Purpose
/api/assignments/:id/grade	POST	Save grade and feedback.
/api/assignments/:id	DELETE	Delete an assignment record.

APPENDIX D: SAMPLE OUTPUT EXPLANATION

D.1 Login Page Output

The login page is the first screen of the project. It displays the project title, a short description of the system, role selection cards for Student and Faculty, username input, password input, remember-me option, and login button. The password field includes a visibility toggle to improve usability during testing.

When a user selects a role, the active role card changes visually. If valid credentials are entered, the user details are stored in browser storage and the user is redirected to the dashboard. If invalid credentials are entered, an error message appears without reloading the page.

D.2 Student Dashboard Output

The student dashboard displays only the assignments that belong to the logged-in student. It contains statistics for total assignments, pending assignments, overdue assignments, and graded assignments. The table shows assignment title, course, due date, status, grade, and available actions.

For pending assignments, the Submit button is available. For submitted or graded assignments, the View button allows the student to open the details modal. This ensures that students can quickly understand what work is due, what has already been submitted, and what feedback has been received.

D.3 Faculty Dashboard Output

The faculty dashboard displays all assignments in the system. Faculty users can view dashboard statistics such as total assignments, assignments needing grading, overdue pending assignments, and average grade. Faculty users can add assignments, view assignment details, grade submitted assignments, and delete records.

The faculty view includes the student name column so that faculty members can identify which student each assignment belongs to. This makes the dashboard useful for monitoring multiple students in one place.

D.4 Modal Window Outputs

Modal	Purpose	Important Fields
Add Assignment	Creates a new assignment record.	Title, course, due date, student name, description
Submit Assignment	Records student submission details.	File name and comments
Grade Assignment	Records faculty evaluation.	Grade and feedback
Details View	Displays complete assignment information.	Status, submission details, grade, feedback

APPENDIX E: VIVA VOCE PREPARATION

E.1 Important Questions and Answers

Question	Answer
What is the main purpose of the project?	The main purpose is to help students and faculty manage assignments, submissions, deadlines, and grading in one place.
Which technologies are used?	HTML, CSS, and JavaScript are used for the frontend. Browser storage is used for prototype persistence.
Why are there separate roles?	Students and faculty perform different tasks, so separate roles make the interface clearer and more secure in concept.
How is assignment data stored?	The prototype stores assignment data in localStorage so records remain after page refresh.
What is sessionStorage used for?	It stores temporary login session details when the user does not choose remember me.
How is overdue status detected?	If an assignment is pending and the due date has passed, the dashboard displays it as overdue.
Can this project be used in real colleges directly?	The prototype demonstrates the workflow, but a real deployment should add backend authentication, database storage, and file upload support.

E.2 Key Points to Explain During Demonstration

- Start by showing the role-based login page.
- Log in as a student and show assignment filtering by logged-in student.
- Submit an assignment and explain how status changes.
- Log out and log in as faculty.
- Add a new assignment for a student.
- Grade a submitted assignment and show the updated status.
- Use search, filter, sort, and details view to demonstrate dashboard usability.
- Explain that localStorage is used for prototype persistence and that a backend is recommended for production.

E.3 Learning Outcomes

Through this project, the team learned how to build a multi-screen web application, design role-based user interfaces, handle form inputs, store data in the browser, update the DOM dynamically, and test user workflows. The project also helped the team understand the importance of clean UI design, validation, and future scalability.

REFERENCES

- MDN Web Docs. HTML documentation. <https://developer.mozilla.org/en-US/docs/Web/HTML>
- MDN Web Docs. CSS documentation. <https://developer.mozilla.org/en-US/docs/Web/CSS>
- MDN Web Docs. JavaScript documentation. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- MDN Web Docs. Web Storage API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- W3Schools. HTML, CSS, and JavaScript tutorials. <https://www.w3schools.com/>
- Visual Studio Code Documentation. <https://code.visualstudio.com/docs>